

Université de Lorraine
UFR Mathématiques et Informatique – Département de Mathématiques



Master 1 de Mathématiques
Année universitaire 2025–2026

Nombres de Salem de trace -3

Mémoire de Master 1

Rédigé par

Ancelle Priscille NAHIMANA Amadou LY

Sous la direction de

Jean-Marc Sac-Épée

Institut Élie Cartan de Lorraine

Université de Lorraine – Metz, France

Table des matières

Introduction	2
1 Notions algébriques	3
1.1 Nombres algébriques, entiers algébriques et polynôme minimal	3
1.2 Trace d'un polynôme et trace d'un nombre algébrique	3
1.3 Polynômes réciproques	3
2 Nombres de Salem	4
2.1 Définition et premières propriétés	4
2.2 La question des traces négatives	6
3 Passage du degré $2d$ au degré d	6
3.1 Le changement de variable $x+1/x$	6
3.2 Localisation des racines de Q	7
4 Reformulation linéaire	7
4.1 Séparateurs de racines	7
4.2 Forme explicite des contraintes	7
4.3 Un problème de faisabilité entière	8
5 La méthode du simplexe	8
5.1 Principe général	8
5.2 Un exemple de résolution par tableaux	9
5.3 Variables artificielles et méthode des deux phases	10
6 Coupes de Gomory et programmation linéaire en nombres entiers	11
6.1 Pourquoi les coupes sont nécessaires	11
6.2 Principe d'une coupe de Gomory	11
6.3 Un exemple de coupes successives	11
7 Application à la recherche de polynômes de Salem	14
7.1 Ce que résout le solveur	14
7.2 Le degré 34	14
7.3 Un exemple de résultat obtenu en degré 34	15
7.4 Explication du calcul	15
7.5 Résultats obtenus par la méthode	16
Conclusion	17
A Programme principal : salem17.jl	18
B Fonctions de résolution : mesFonctionsSalem.jl	20

Introduction

Un nombre de Salem est un entier algébrique réel strictement supérieur à 1, dont tous les conjugués ont un module inférieur ou égal à 1, avec l'un au moins des conjugués de module 1.

Ce mémoire porte sur la recherche de nombres de Salem de trace -3 , la trace d'un entier algébrique étant par définition celle de son polynôme minimal, c'est-à-dire la somme des racines de ce polynôme.

Si $P(x) = x^n + a_{n-1}x^{n-1} + \dots + a_0$ est un polynôme unitaire, la somme de ses racines vaut $-a_{n-1}$. Prescrire la trace -3 pour un nombre de Salem revient donc à imposer le coefficient $a_{n-1} = 3$ à son polynôme minimal.

Les traces positives se construisent sans difficulté : pour $n \geq 1$, le plus grand zéro réel de $x^4 - nx^3 - (2n+1)x^2 - nx + 1$ est un nombre de Salem de trace n . Les traces négatives sont plus difficiles à exhiber. C. J. Smyth a étudié le cas de la trace -1 , puis J. McKee et C. J. Smyth ont traité la trace -2 et établi l'existence de nombres de Salem de toutes traces entières [3, 4, 5]. Pour la trace -3 , le premier exemple connu est dû à J. McKee, qui a exhibé un nombre de Salem de trace -3 et de degré 54 [8]. Cependant, il est bien connu que le degré minimal d'un polynôme de trace -3 est 32 [10]. On voit donc que le degré 54 de McKee est loin de cette borne inférieure.

L'idée développée ici est de remplacer la recherche directe du polynôme de Salem par un problème de dimension moitié. Si P est réciproque de degré $2d$, on peut écrire

$$P(x) = x^d Q\left(x + \frac{1}{x}\right),$$

où Q est un polynôme unitaire de degré d à racines toutes réelles. Les conjugués de P situés sur le cercle unité (on démontre que c'est le cas pour toutes les racines de P , sauf 2) donnent alors des racines de Q dans l'intervalle $(-2, 2)$, tandis que la racine de Salem et son inverse donnent une racine de Q strictement supérieure à 2.

On passe ainsi au problème suivant : chercher un polynôme unitaire $Q \in \mathbb{Z}[y]$, de trace -3 , ayant $d-1$ racines dans $(-2, 2)$ et une racine dans $(2, R)$. Pour le degré 34, on peut prendre $R = 4.69$ [9], valeur qui majore les racines (toutes réelles) de Q . Pour un choix fixé de séparateurs entre ces racines, l'existence d'une racine dans chaque intervalle est en imposant une alternance de signes au polynôme Q . Or la valeur de Q en un point fixé est une expression linéaire en ses coefficients. La recherche devient donc un problème de faisabilité en programmation linéaire en nombres entiers.

L'utilisation du simplexe (avec introduction de coupes de Gomory) est introduite dans ce contexte de programmation linéaire en entiers. Le simplexe résout les relaxations linéaires qui apparaissent lorsque l'on oublie temporairement que les coefficients doivent être entiers. Les coupes, le branchement et les autres mécanismes d'un solveur entier servent ensuite à éliminer les solutions fractionnaires sans perdre les solutions entières. Le calcul fournit seulement des candidats ; la validation finale reste algébrique : on reconstruit P , on vérifie sa réciprocity, sa trace, la position de ses racines et son irréductibilité.

Le mémoire est organisé de la manière suivante. Les premières sections fixent les notions algébriques utilisées : trace, polynôme réciproque, nombre de Salem. On explique ensuite la réduction du degré $2d$ au degré d , puis la traduction des conditions sur les racines en inégalités linéaires. Les sections suivantes rappellent la méthode du simplexe et les coupes de Gomory sur des exemples explicites. Enfin, on applique ces outils à la recherche de nombres de Salem de trace -3 , en particulier au degré 34. Les programmes Julia utilisés sont reproduits en annexe.

1 Notions algébriques

1.1 Nombres algébriques, entiers algébriques et polynôme minimal

Définition 1.1. Un nombre complexe α est dit *algébrique* s'il existe un polynôme non nul $P \in \mathbb{Z}[x]$ tel que $P(\alpha) = 0$.

Par exemple, $\sqrt{2}$ est algébrique, puisqu'il annule $x^2 - 2$, et i est algébrique, puisqu'il annule $x^2 + 1$.

Définition 1.2. Un nombre complexe α est un *entier algébrique* s'il est racine d'un polynôme unitaire à coefficients entiers.

La condition "unitaire" est essentielle. Le nombre $1/2$ est algébrique, car il est racine de $2x - 1$, mais il n'est pas un entier algébrique.

Définition 1.3. Soit α un nombre algébrique. Son *polynôme minimal* sur $\mathbb{Q}$ est l'unique polynôme unitaire irréductible $P_\alpha \in \mathbb{Q}[x]$ tel que $P_\alpha(\alpha) = 0$.

Les racines de P_α sont appelées les *conjugués algébriques* de α . Lorsque α est un entier algébrique, son polynôme minimal appartient en fait à $\mathbb{Z}[x]$.

Exemple 1.4. Le polynôme minimal de $\sqrt{2}$ est $x^2 - 2$. Les conjugués de $\sqrt{2}$ sont donc $\sqrt{2}$ et $-\sqrt{2}$.

1.2 Trace d'un polynôme et trace d'un nombre algébrique

Soit

$$P(x) = a_d x^d + a_{d-1} x^{d-1} + \cdots + a_0$$

un polynôme de degré d , de racines $\alpha_1, \dots, \alpha_d$, comptées avec multiplicité. Sa trace est

$$\text{Tr}(P) = \alpha_1 + \cdots + \alpha_d.$$

La formule de Viète donne

$$\text{Tr}(P) = -\frac{a_{d-1}}{a_d}.$$

En particulier, si P est unitaire, alors

$$\text{Tr}(P) = -a_{d-1}.$$

Définition 1.5. La trace d'un nombre algébrique α est la trace de son polynôme minimal P_α .

Exemple 1.6. Pour

$$P(x) = x^2 - 5x + 6 = (x - 2)(x - 3),$$

les racines sont 2 et 3, donc $\text{Tr}(P) = 5$. On retrouve bien $-a_1 = 5$.

1.3 Polynômes réciproques

Définition 1.7. Soit $P \in \mathbb{Z}[x]$ un polynôme de degré n . On appelle *polynôme réciproque* de P le polynôme

$$P^*(x) = x^n P\left(\frac{1}{x}\right).$$

On dit que P est *réciproque* si $P = P^*$.

Si

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

alors

$$P^*(x) = a_0 x^n + a_1 x^{n-1} + \cdots + a_{n-1} x + a_n.$$

Un polynôme est donc réciproque lorsque sa suite de coefficients est symétrique.

Exemple 1.8. Le polynôme

$$P(x) = x^4 - 7x^3 + 6x^2 + 5x + 2$$

a pour polynôme réciproque

$$P^*(x) = 2x^4 + 5x^3 + 6x^2 - 7x + 1.$$

Il n'est donc pas réciproque.

La réciprocité a une conséquence immédiate sur les racines : si $\alpha \neq 0$ est racine d'un polynôme réciproque, alors $1/\alpha$ est également racine.

2 Nombres de Salem

2.1 Définition et premières propriétés

Définition 2.1. Un *nombre de Salem* est un entier algébrique réel $\tau > 1$ tel que tous ses conjugués ont un module inférieur ou égal à 1, et tel qu'au moins un de ses conjugués a un module exactement égal à 1.

L'exemple le plus célèbre est le nombre de Salem

$$\tau_0 = 1.176280\dots,$$

dont le polynôme minimal (polynôme de Lehmer) est

$$L(x) = x^{10} + x^9 - x^7 - x^6 - x^5 - x^4 - x^3 + x + 1.$$

Proposition 2.2. *Le polynôme minimal d'un nombre de Salem est réciproque. En particulier, si τ est un nombre de Salem, alors $1/\tau$ est un conjugué de τ , et tous les autres conjugués, distincts de τ et de $1/\tau$, sont de module 1.*

Démonstration. Soit $P \in \mathbb{Z}[x]$ le polynôme minimal du nombre de Salem τ , et soit $d = \deg P$. Montrons que P est réciproque.

Par définition, P possède au moins une racine ζ située sur le cercle unité. Comme P est à coefficients réels, $\bar{\zeta}$ est également racine de P . De plus, puisque $|\zeta| = 1$, on a

$$\bar{\zeta} = \zeta^{-1}.$$

Ainsi, ζ^{-1} est aussi racine de P .

Considérons le polynôme réciproque

$$P^*(x) = x^d P(1/x).$$

Ses racines sont les inverses des racines de P . Comme ζ^{-1} est racine de P , il s'ensuit que ζ est racine de P^* . Les polynômes P et P^* ont donc une racine commune.

Or P est irréductible sur $\mathbb{Q}$. Il divise donc P^* dans $\mathbb{Q}[x]$. Comme P et P^* ont le même degré, il existe une constante $c \in \mathbb{Q}$ telle que

$$P^*(x) = cP(x).$$

En comparant les coefficients dominant et constant, on obtient $c = \pm 1$.

Le cas $P^* = -P$ est impossible : en évaluant en $x = 1$, on aurait

$$P(1) = -P(1),$$

donc $P(1) = 0$, ce qui contredirait le fait que P est le polynôme minimal de $\tau > 1$. Par conséquent,

$$P^*(x) = P(x),$$

c'est-à-dire

$$x^d P(1/x) = P(x).$$

Ainsi, le polynôme minimal P est réciproque.

Comme P est réciproque, on a

$$P(x) = x^d P(1/x).$$

En particulier, puisque $P(\tau) = 0$ et $\tau \neq 0$, on obtient

$$0 = P(\tau) = \tau^d P(1/\tau),$$

d'où

$$P(1/\tau) = 0.$$

Ainsi, $1/\tau$ est bien une racine de P , donc un conjugué de τ .

Il reste à préciser le module des autres conjugués. Par définition d'un nombre de Salem, tous les conjugués de τ , autres que τ lui-même, sont de module au plus 1. Soit donc α un conjugué de τ distinct de τ et de $1/\tau$. On sait déjà que

$$|\alpha| \leq 1.$$

Supposons que $|\alpha| < 1$. Comme P est réciproque, l'inverse α^{-1} est aussi racine de P , donc aussi un conjugué de τ . Mais alors

$$|\alpha^{-1}| > 1.$$

Or le seul conjugué de τ de module strictement supérieur à 1 est τ lui-même. On aurait donc $\alpha^{-1} = \tau$, c'est-à-dire $\alpha = 1/\tau$, contradiction.

Ainsi $|\alpha|$ ne peut pas être strictement inférieur à 1. Comme on avait déjà $|\alpha| \leq 1$, il vient

$$|\alpha| = 1.$$

Tous les conjugués de τ distincts de τ et de $1/\tau$ sont donc de module 1. □

Proposition 2.3. *Le degré du polynôme minimal d'un nombre de Salem est pair et au moins égal à 4.*

Démonstration. Les racines se regroupent par paires $\alpha, 1/\alpha$. Le polynôme minimal contient en particulier les deux racines réelles τ et $1/\tau$. Il contient aussi au moins une paire de racines complexes conjuguées situées sur le cercle unité. Le degré est donc pair et au moins égal à 4. □

2.2 La question des traces négatives

La trace d'un nombre de Salem est la somme de tous ses conjugués.

On peut exhiber des nombres de Salem de trace positive à l'aide d'une construction simple : pour tout $n \geq 1$, le polynôme

$$x^4 - nx^3 - (2n+1)x^2 - nx + 1$$

fournit un nombre de Salem de trace n .

Un exemple de polynôme de Salem de trace nulle est donné par

$$x^6 - x^4 - 2x^3 - x^2 + 1.$$

Les traces négatives constituent un problème plus difficile, comme on peut le voir dans les travaux de Smyth, puis de McKee et Smyth [3, 4, 5].

L'approche suivie dans ce mémoire consiste à transférer cette recherche vers la recherche d'un polynôme auxiliaire à racines toutes réelles, de degré moitié. Ce transfert remplace la géométrie des racines de P , situées dans le plan complexe, par une condition de racines réelles pour un polynôme Q . C'est cette forme réelle qui permet l'utilisation de la programmation linéaire en nombres entiers.

3 Passage du degré 2d au degré d

3.1 Le changement de variable $x+1/x$

Soit P un polynôme réciproque unitaire de degré pair $2d$. Il s'écrit sous la forme

$$P(x) = x^{2d} + c_1x^{2d-1} + c_2x^{2d-2} + \cdots + c_dx^d + \cdots + c_2x^2 + c_1x + 1.$$

En factorisant par x^d , on obtient

$$P(x) = x^d \left[(x^d + x^{-d}) + c_1(x^{d-1} + x^{-(d-1)}) + \cdots + c_{d-1}(x + x^{-1}) + c_d \right].$$

Posons

$$y = x + \frac{1}{x}.$$

Pour tout entier $m \geq 0$, l'expression $x^m + x^{-m}$ est un polynôme en y . Plus précisément, si

$$T_0(y) = 2, \quad T_1(y) = y, \quad T_{m+1}(y) = yT_m(y) - T_{m-1}(y),$$

alors

$$T_m\left(x + \frac{1}{x}\right) = x^m + x^{-m}.$$

Les premiers exemples sont

$$T_2(y) = y^2 - 2, \quad T_3(y) = y^3 - 3y.$$

Il existe donc un polynôme unitaire $Q \in \mathbb{Z}[y]$, de degré d , tel que

$$P(x) = x^d Q\left(x + \frac{1}{x}\right).$$

Exemple 3.1. Le polynôme de Lehmer vérifie

$$L(x) = x^5 Q_0\left(x + \frac{1}{x}\right),$$

où

$$Q_0(y) = y^5 + y^4 - 5y^3 - 5y^2 + 4y + 3.$$

Ce polynôme Q_0 possède quatre racines dans $(-2, 2)$ et une racine strictement supérieure à 2.

3.2 Localisation des racines de Q

Si α est une racine de module 1, on peut écrire $\alpha = e^{i\theta}$, et alors

$$\alpha + \frac{1}{\alpha} = 2 \cos \theta.$$

Lorsque $\alpha \neq \pm 1$, cette quantité appartient à $(-2, 2)$. Les racines ± 1 ne peuvent pas apparaître dans le polynôme minimal irréductible d'un nombre de Salem de degré supérieur à 1. Les conjugués de module 1 donnent donc $d - 1$ racines de Q dans $(-2, 2)$.

En revanche, si $\tau > 1$ est la racine de Salem, alors

$$\tau + \frac{1}{\tau} > 2.$$

Ainsi, un polynôme de Salem P de degré $2d$ donne un polynôme unitaire Q de degré d tel que

$$P(x) = x^d Q\left(x + \frac{1}{x}\right),$$

avec $d - 1$ racines dans $(-2, 2)$ et une racine strictement supérieure à 2.

4 Reformulation linéaire

4.1 Séparateurs de racines

Soit

$$Q(u) = u^d + a_{d-1}u^{d-1} + \cdots + a_0$$

un polynôme unitaire de degré d . Supposons que l'on veuille imposer une racine dans chacun des intervalles

$$(t_0, t_1), (t_1, t_2), \dots, (t_{d-1}, t_d),$$

où

$$t_0 < t_1 < \cdots < t_d.$$

Si Q change de signe entre deux séparateurs consécutifs, le théorème des valeurs intermédiaires assure l'existence d'une racine dans l'intervalle correspondant.

Si l'on impose une racine dans chacun des d intervalles, les signes aux séparateurs doivent donc alterner. On peut écrire cette alternance sous la forme

$$(-1)^{d-j} Q(t_j) > 0, \quad j = 0, \dots, d.$$

Pour des séparateurs fixés, chaque quantité $Q(t_j)$ est linéaire en les coefficients inconnus de Q . Une condition de localisation des racines est donc remplacée par des inégalités linéaires.

4.2 Forme explicite des contraintes

Dans le cas de la trace -3 , on sait que

$$a_{d-1} = 3.$$

Les inconnues sont donc

$$a_0, a_1, \dots, a_{d-2}.$$

Pour un séparateur t_j , on a

$$Q(t_j) = t_j^d + 3t_j^{d-1} + \sum_{i=0}^{d-2} a_i t_j^i.$$

La contrainte d'alternance devient

$$(-1)^{d-j} \sum_{i=0}^{d-2} a_i t_j^i > -(-1)^{d-j} (t_j^d + 3t_j^{d-1}).$$

C'est une inégalité linéaire en $a_0, \dots, a_{d-2}$.

Dans le programme, les inégalités strictes sont remplacées par des inégalités non strictes avec une marge numérique positive. Cette marge évite de retenir des solutions où un séparateur serait presque racine du polynôme, situation instable numériquement.

4.3 Un problème de faisabilité entière

Il n'y a pas de fonction naturelle à optimiser. Le but est seulement de savoir s'il existe un vecteur entier

$$(a_0, \dots, a_{d-2}) \in \mathbb{Z}^{d-1}$$

qui vérifie les contraintes de signe et les bornes choisies sur les coefficients. On obtient donc un problème de faisabilité en programmation linéaire en nombres entiers.

Les bornes sur les coefficients sont obtenues à partir d'expériences numériques : on génère des racines réelles dans les intervalles voulus, on reconstruit les polynômes correspondants, puis on observe les ordres de grandeur des coefficients. Ces bornes définissent une région de recherche suffisamment large pour contenir des exemples, mais suffisamment limitée pour que le solveur soit efficace. La procédure utilisée est la suivante.

1. Choisir un degré d et la borne supérieure correspondante $R > 2$ pour la racine extérieure de Q .
2. Tirer $d - 2$ séparateurs dans $(-2, 2)$, puis ajouter les séparateurs $-2, 2$ et R .
3. Écrire les contraintes d'alternance de signes aux séparateurs.
4. Ajouter les contraintes de trace et les bornes sur les coefficients.
5. Résoudre le problème de faisabilité entière obtenu.
6. Si un polynôme Q est trouvé, reconstruire $P(x) = x^d Q(x + 1/x)$.
7. Vérifier exactement l'irréductibilité de P et numériquement la position de ses racines.
8. En l'absence de solution, recommencer avec un autre choix de séparateurs.

La partie aléatoire intervient seulement dans le choix des séparateurs.

5 La méthode du simplexe

5.1 Principe général

La méthode du simplexe résout des problèmes de programmation linéaire de la forme

$$\begin{array}{ll} \text{maximiser} & c^T x \\ \text{sous les contraintes} & Ax \leq b, \\ & x \geq 0. \end{array}$$

Les contraintes définissent un polyèdre. Lorsqu'une fonction linéaire atteint un maximum sur un polyèdre non vide et borné dans la direction considérée, elle l'atteint en un sommet. Le simplexe exploite cette propriété : il passe d'un sommet à un sommet adjacent en améliorant la valeur de la fonction objectif.

Pour mettre le problème sous forme de tableau, on introduit des variables d'écart. Une contrainte

$$a_1 x_1 + \dots + a_n x_n \leq b$$

devient

$$a_1x_1 + \dots + a_nx_n + e = b, \quad e \geq 0.$$

Les variables d'écart fournissent souvent une base initiale. Une opération de pivot fait entrer une variable dans la base et en fait sortir une autre. Dans la convention utilisée ci-dessous, tant qu'un coefficient strictement positif apparaît dans la dernière ligne du tableau, la valeur de la fonction objectif peut encore être améliorée.

À chaque étape :

1. on choisit une colonne de pivot correspondant à une amélioration possible ;
2. on choisit la ligne du pivot par le test des rapports ;
3. on effectue un pivot de Gauss-Jordan ;
4. on recommence jusqu'à ce que la dernière ligne ne contienne plus de coefficient positif.

Le pivot de Gauss-Jordan peut être décrit ainsi.

```
1 fonction pivot(M, ligne, colonne)
2   diviser la ligne du pivot par le coefficient pivot
3   pour chaque autre ligne :
4     enlever un multiple de la ligne pivot
5     afin d'annuler le coefficient dans la colonne du pivot
6 fin
```

5.2 Un exemple de résolution par tableaux

Considérons le problème

$$\text{maximiser } F(x_1, x_2) = x_1 + 2x_2$$

sous les contraintes

$$\begin{cases} -3x_1 + 2x_2 \leq 2, \\ -x_1 + 2x_2 \leq 1, \\ x_1 + x_2 \leq 5, \\ x_1, x_2 \geq 0. \end{cases}$$

On introduit trois variables d'écart $e_1, e_2, e_3 \geq 0$:

$$\begin{cases} -3x_1 + 2x_2 + e_1 = 2, \\ -x_1 + 2x_2 + e_2 = 1, \\ x_1 + x_2 + e_3 = 5. \end{cases}$$

La fonction objectif s'écrit

$$F - x_1 - 2x_2 = 0.$$

Le tableau initial est donc

$$M = \begin{pmatrix} -3 & 2 & 1 & 0 & 0 & 2 \\ -1 & 2 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 5 \\ 1 & 2 & 0 & 0 & 0 & 0 \end{pmatrix},$$

avec les colonnes

$$x_1, x_2, e_1, e_2, e_3 \mid b.$$

Dans la dernière ligne, le coefficient positif le plus grand est 2, dans la colonne de x_2 . On calcule les rapports sur les lignes où le coefficient de cette colonne est positif :

$$\frac{2}{2} = 1, \quad \frac{1}{2} = \frac{1}{2}, \quad \frac{5}{1} = 5.$$

Le plus petit rapport est $1/2$. Le pivot se trouve donc sur la deuxième ligne, dans la colonne de x_2 . Après pivot, on obtient

$$N = \begin{pmatrix} -2 & 0 & 1 & -1 & 0 & 1 \\ -\frac{1}{2} & 1 & 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ \frac{3}{2} & 0 & 0 & -\frac{1}{2} & 1 & \frac{9}{2} \\ 2 & 0 & 0 & -1 & 0 & -1 \end{pmatrix}.$$

La dernière ligne contient encore un coefficient positif, 2, dans la colonne de x_1 . Dans cette colonne, le seul coefficient strictement positif utilisable est $3/2$, situé sur la troisième ligne. On pivote donc sur cette entrée. Le tableau final est

$$N = \begin{pmatrix} 0 & 0 & 1 & -\frac{5}{3} & \frac{4}{3} & 7 \\ 0 & 1 & 0 & \frac{1}{3} & \frac{1}{3} & 2 \\ 1 & 0 & 0 & -\frac{1}{3} & \frac{2}{3} & 3 \\ 0 & 0 & 0 & -\frac{1}{3} & -\frac{4}{3} & -7 \end{pmatrix}.$$

Il n'y a plus de coefficient positif dans la dernière ligne. On lit donc les variables de base :

$$x_1 = 3, \quad x_2 = 2.$$

La valeur maximale est

$$F_{\max} = 3 + 2 \times 2 = 7.$$

Ainsi

$$\boxed{x_1 = 3, \quad x_2 = 2, \quad F_{\max} = 7.}$$

Cet exemple ne sert pas seulement à illustrer un algorithme de calcul. Il montre ce que fait un solveur lorsqu'il étudie une relaxation linéaire : il parcourt les sommets d'un polyèdre défini par des inégalités linéaires.

5.3 Variables artificielles et méthode des deux phases

L'exemple précédent est favorable : les contraintes sont de type $\leq$, et les variables d'écart donnent immédiatement une base réalisable. Si une contrainte est de type $\geq$, par exemple

$$x_1 + x_2 \geq 10,$$

on introduit une variable d'excès et une variable artificielle :

$$x_1 + x_2 - e_1 + a_1 = 10.$$

La méthode des deux phases consiste d'abord à résoudre un problème auxiliaire dont l'objectif est d'annuler les variables artificielles. Si cette première phase se termine avec une valeur nulle, on dispose d'une solution réalisable du problème initial ; on peut alors lancer la deuxième phase avec la vraie fonction objectif.

Cette distinction est importante pour notre application. Avant de chercher à optimiser quoi que ce soit, le solveur doit d'abord établir l'existence d'un point satisfaisant toutes les contraintes de signe et d'intégralité.

6 Coupes de Gomory et programmation linéaire en nombres entiers

6.1 Pourquoi les coupes sont nécessaires

Le simplexe résout un problème linéaire continu. Dans notre recherche de polynômes de Salem, les coefficients doivent être entiers.

La stratégie générale consiste à résoudre d'abord la relaxation continue, c'est-à-dire le problème obtenu en supprimant la contrainte $x \in \mathbb{Z}^n$. Si la solution obtenue est entière, elle convient aussi au problème entier. Si elle est fractionnaire, il faut ajouter des contraintes qui l'éliminent, tout en conservant toutes les solutions entières possibles. Les coupes de Gomory fournissent un exemple explicite de ce mécanisme.

6.2 Principe d'une coupe de Gomory

On part d'un tableau optimal du problème relaxé. Supposons qu'une ligne du tableau s'écrive

$$x_B + a_1 y_1 + a_2 y_2 + \cdots + a_k y_k = b,$$

où x_B est une variable de base qui doit être entière, les y_j sont des variables hors base qui doivent aussi être entières, et b n'est pas entier.

On note

$$\{r\} = r - \lfloor r \rfloor$$

la partie fractionnaire de r . La coupe de Gomory associée à cette ligne est

$$\boxed{\{a_1\} y_1 + \{a_2\} y_2 + \cdots + \{a_k\} y_k \geq \{b\} .}$$

Elle est satisfaite par toutes les solutions entières du problème initial, mais elle exclut la solution fractionnaire courante. Elle réduit donc la relaxation sans supprimer les solutions entières recherchées.

6.3 Un exemple de coupes successives

Considérons le problème en nombres entiers

$$\text{maximiser } z = 10x_1 + 11x_2$$

sous les contraintes

$$\begin{cases} 10x_1 + 12x_2 \leq 59, \\ x_1, x_2 \geq 0, \\ x_1, x_2 \in \mathbb{Z}. \end{cases}$$

Résolution du problème relaxé

On commence par supprimer la contrainte d'intégrité. On introduit une variable d'écart $e_1 \geq 0$:

$$10x_1 + 12x_2 + e_1 = 59.$$

Le tableau initial est

$$M = \begin{pmatrix} 10 & 12 & 1 & 59 \\ 10 & 11 & 0 & 0 \end{pmatrix},$$

avec les colonnes

$$x_1, x_2, e_1 \mid b.$$

La méthode du simplexe donne le tableau final

$$N = \begin{pmatrix} 1 & \frac{6}{5} & \frac{1}{10} & \frac{59}{10} \\ 0 & -1 & -1 & -59 \end{pmatrix}.$$

On lit

$$x_1 = \frac{59}{10}, \quad x_2 = 0, \quad z = 59.$$

La solution optimale relaxée n'est pas entière.

Première coupe de Gomory

La première ligne du tableau final est

$$x_1 + \frac{6}{5}x_2 + \frac{1}{10}e_1 = \frac{59}{10}.$$

Le second membre a pour partie fractionnaire

$$\left\{ \frac{59}{10} \right\} = \frac{9}{10}.$$

Les coefficients des variables hors base ont pour parties fractionnaires

$$\left\{ \frac{6}{5} \right\} = \frac{1}{5}, \quad \left\{ \frac{1}{10} \right\} = \frac{1}{10}.$$

La coupe de Gomory est donc

$$\frac{1}{5}x_2 + \frac{1}{10}e_1 \geq \frac{9}{10}.$$

En multipliant par 10, on obtient

$$2x_2 + e_1 \geq 9.$$

Or

$$e_1 = 59 - 10x_1 - 12x_2.$$

Donc

$$2x_2 + 59 - 10x_1 - 12x_2 \geq 9,$$

c'est-à-dire

$$x_1 + x_2 \leq 5.$$

La première coupe est donc

$$\boxed{x_1 + x_2 \leq 5.}$$

Elle élimine bien la solution fractionnaire précédente, car

$$\frac{59}{10} + 0 = 5.9 > 5.$$

On ajoute cette contrainte. Le nouveau système relaxé est

$$\begin{cases} 10x_1 + 12x_2 + e_1 = 59, \\ x_1 + x_2 + e_2 = 5. \end{cases}$$

Après application du simplexe, on obtient le tableau

$$N = \begin{pmatrix} 0 & 1 & \frac{1}{2} & -5 & \frac{9}{2} \\ 1 & 0 & -\frac{1}{2} & 6 & \frac{1}{2} \\ 0 & 0 & -\frac{1}{2} & -5 & -\frac{109}{2} \end{pmatrix}.$$

On lit

$$x_1 = \frac{1}{2}, \quad x_2 = \frac{9}{2}, \quad z = \frac{109}{2}.$$

La solution n'est toujours pas entière.

Seconde coupe de Gomory

On utilise la première ligne du dernier tableau :

$$x_2 + \frac{1}{2}e_1 - 5e_2 = \frac{9}{2}.$$

La partie fractionnaire du second membre est

$$\left\{ \frac{9}{2} \right\} = \frac{1}{2}.$$

Les coefficients hors base donnent

$$\left\{ \frac{1}{2} \right\} = \frac{1}{2}, \quad \{-5\} = 0.$$

La coupe est donc

$$\frac{1}{2}e_1 \geq \frac{1}{2},$$

c'est-à-dire

$$e_1 \geq 1.$$

Comme

$$e_1 = 59 - 10x_1 - 12x_2,$$

on obtient

$$10x_1 + 12x_2 \leq 58.$$

La seconde coupe de Gomory est donc

$$\boxed{10x_1 + 12x_2 \leq 58.}$$

Elle élimine la solution précédente car, pour $(x_1, x_2) = (1/2, 9/2)$,

$$10x_1 + 12x_2 = 5 + 54 = 59 > 58.$$

En ajoutant cette seconde coupe, puis en appliquant à nouveau le simplexe, on obtient finalement

$$x_1 = 1, \quad x_2 = 4, \quad z = 54.$$

La solution entière optimale est donc

$$\boxed{x_1 = 1, \quad x_2 = 4, \quad z_{\max} = 54.}$$

Cet exemple montre le rôle des coupes : elles n'ont pas pour but de résoudre seules le problème entier, mais de renforcer progressivement la relaxation linéaire. Dans les solveurs modernes, elles sont combinées avec des procédures de branchement.

7 Application à la recherche de polynômes de Salem

7.1 Ce que résout le solveur

Pour un choix fixé de séparateurs, le solveur manipule les coefficients du polynôme auxiliaire Q , soumis à des inégalités linéaires.

Le modèle obtenu est de la forme

$$Aa \leq b, \quad \ell_i \leq a_i \leq u_i, \quad a_i \in \mathbb{Z}.$$

Le vecteur a contient les coefficients inconnus. Le solveur commence par examiner des relaxations où les contraintes $a_i \in \mathbb{Z}$ sont remplacées par $a_i \in \mathbb{R}$. Ces relaxations relèvent exactement de la programmation linéaire continue, donc du simplexe ou de méthodes voisines.

Si une relaxation est infaisable, les séparateurs choisis ne peuvent pas produire de polynôme dans les bornes fixées. Si elle est faisable mais fractionnaire, le solveur utilise le branchement et des coupes pour imposer l'intégralité. Le résultat d'un appel au solveur est donc soit une preuve d'infaisabilité pour le modèle courant, soit un vecteur entier de coefficients à tester algébriquement.

7.2 Le degré 34

Pour chercher un nombre de Salem de degré 34, on prend $2d = 34$, donc $d = 17$. Le polynôme auxiliaire est de la forme

$$Q(y) = y^{17} + 3y^{16} + a_{15}y^{15} + a_{14}y^{14} + \cdots + a_1y + a_0.$$

Le coefficient de y^{16} est 3, car la trace de Q doit être -3 .

Les inconnues sont donc les seize entiers

$$a_0, a_1, \dots, a_{15}.$$

On cherche Q avec seize racines dans $(-2, 2)$ et une racine dans $(2, R)$. Les calculs utilisent les bornes suivantes [9] pour la racine extérieure :

d	$\text{Tr}(Q)$	intervalle contenant les racines de Q
17	-3	$(-2, 4.69)$
18	-3	$(-2, 5.11)$
19	-3	$(-2, 5.50)$
20	-3	$(-2, 5.86)$

Pour $d = 17$, on prend donc $R = 4.69$.

On tire quinze séparateurs dans $(-2, 2)$, que l'on ordonne, puis on ajoute $-2, 2$ et 4.69 :

$$-2 = t_0 < t_1 < \cdots < t_{15} < t_{16} = 2 < t_{17} = 4.69.$$

On impose

$$(-1)^{17-j}Q(t_j) > 0, \quad j = 0, \dots, 17.$$

Ces dix-huit contraintes sont linéaires en $a_0, \dots, a_{15}$.

Par exemple, au point $t_0 = -2$, on obtient

$$-Q(-2) > 0.$$

Au point $t_{17} = 4.69$, on obtient

$$Q(4.69) > 0,$$

c'est-à-dire

$$4.69^{17} + 3 \cdot 4.69^{16} + \sum_{i=0}^{15} a_i 4.69^i > 0.$$

C'est encore une contrainte linéaire en $a_0, \dots, a_{15}$.

7.3 Un exemple de résultat obtenu en degré 34

L'un des polynômes auxiliaires obtenus par cette méthode, écrit dans la variable non décalée $y = x + x^{-1}$, est

$$\begin{aligned} Q(y) = & y^{17} + 3y^{16} - 15y^{15} - 58y^{14} + 49y^{13} \\ & + 363y^{12} + 109y^{11} - 966y^{10} - 833y^9 \\ & + 1061y^8 + 1460y^7 - 246y^6 - 894y^5 \\ & - 186y^4 + 139y^3 + 45y^2 - 2y - 1. \end{aligned}$$

Sa trace vaut bien -3 , puisque le coefficient de y^{16} est 3 .

Numériquement, ses racines sont approximativement :

1	-1.937762	10	-0.188161
2	-1.907272	11	0.148786
3	-1.852331	12	0.450604
4	-1.703455	13	0.944049
5	-1.517572	14	1.446109
6	-1.373628	15	1.566127
7	-1.189902	16	1.935545
8	-0.578317	17	3.123748
9	-0.366568		

Seize racines sont donc dans $(-2, 2)$, et la dernière appartient à $(2, 4.69)$.

En appliquant le changement de variable inverse,

$$P(x) = x^{17} Q\left(x + \frac{1}{x}\right),$$

on obtient le polynôme réciproque

$$\begin{aligned} P(x) = & x^{34} + 3x^{33} + 2x^{32} - 10x^{31} - 40x^{30} \\ & - 89x^{29} - 149x^{28} - 208x^{27} - 257x^{26} - 293x^{25} \\ & - 315x^{24} - 322x^{23} - 311x^{22} - 281x^{21} - 237x^{20} \\ & - 191x^{19} - 156x^{18} - 143x^{17} - 156x^{16} - 191x^{15} \\ & - 237x^{14} - 281x^{13} - 311x^{12} - 322x^{11} - 315x^{10} \\ & - 293x^9 - 257x^8 - 208x^7 - 149x^6 - 89x^5 \\ & - 40x^4 - 10x^3 + 2x^2 + 3x + 1. \end{aligned}$$

Le coefficient de x^{33} vaut 3 , donc $\text{Tr}(P) = -3$. La symétrie des coefficients montre que P est réciproque.

La racine réelle strictement supérieure à 1 vaut approximativement

$$\tau = 2.7616448085 \dots$$

7.4 Explication du calcul

Dans les faits, on ne connaît pas les coefficients de Q . On choisit aléatoirement des séparateurs, en espérant qu'ils vont séparer les racines de Q . L'alternance de signes impose alors un système linéaire décrivant un polyèdre dans l'espace des coefficients $(a_0, \dots, a_{15})$.

Le solveur cherche un point entier de ce polyèdre. La relaxation continue donne un premier problème linéaire ; si cette relaxation est infaisable, le choix de séparateurs est rejeté. Si elle est faisable, le solveur impose l'intégralité par branchement et par coupes. Les coupes de Gomory

étudiées plus haut donnent le modèle de cette opération : exclure une solution fractionnaire sans exclure les solutions entières.

Une fois un vecteur entier trouvé, le calcul d'optimisation est terminé. Il faut alors encore reconstruire le polynôme réciproque, contrôler la position de ses racines et vérifier son irréductibilité.

7.5 Résultats obtenus par la méthode

Les calculs consacrés aux petits degrés de trace -3 cherchent des polynômes auxiliaires de degrés 17, 18, 19 et 20. Après le changement de variable inverse, ils donnent des polynômes de Salem de degrés 34, 36, 38 et 40. Les nombres de polynômes auxiliaires trouvés sont les suivants :

deg Q	deg P	nombre de polynômes listés
17	34	2
18	36	4
19	38	14
20	40	45

Dans la convention adoptée ici, tous ces polynômes auxiliaires ont trace -3 . Chaque ligne fournit, après transformation inverse, un polynôme réciproque de degré double et de trace -3 .

On rencontre aussi la même stratégie avec la variable décalée $u = y + 2$. Elle transporte simplement l'intervalle $(-2, 2)$ sur $(0, 4)$ et l'intervalle extérieur $(2, R)$ sur $(4, R + 2)$. Dans ce mémoire, on garde la convention non décalée $y = x + x^{-1}$, afin que l'écriture du polynôme réciproque reste toujours

$$P(x) = x^d Q\left(x + \frac{1}{x}\right).$$

La structure algorithmique est inchangée : séparateurs, alternance de signes, contraintes linéaires, recherche entière, puis vérification algébrique.

Conclusion

La recherche de nombres de Salem de trace -3 repose sur deux idées. La première est algébrique : la réciprocité du polynôme minimal permet de remplacer un polynôme de degré $2d$ par un polynôme auxiliaire de degré d . Avec le changement de variable

$$x + \frac{1}{x},$$

les conjugués situés sur le cercle unité deviennent des racines dans $(-2, 2)$, tandis que la racine de Salem devient une racine strictement supérieure à 2. Dans cette convention, la condition de trace -3 devient simplement $\text{Tr}(Q) = -3$.

La seconde idée est algorithmique : on choisit des séparateurs et l'on impose une alternance de signes. Pour des séparateurs fixés, ces contraintes sont linéaires en les coefficients du polynôme. La recherche devient donc un problème de faisabilité en programmation linéaire en nombres entiers. Le simplexe intervient dans les relaxations linéaires ; les coupes et le branchement imposent ensuite l'intégralité des coefficients.

Le solveur fournit des candidats ; il faut ensuite reconstruire le polynôme réciproque, vérifier sa trace, contrôler la position de ses racines et établir son irréductibilité. C'est cette combinaison entre réduction algébrique, optimisation entière et vérification exacte qui permet de produire des exemples de nombres de Salem de trace -3 en petit degré.

A Programme principal : salem17.jl

Le programme suivant pilote la recherche en degré réduit $d = 17$. Il génère des séparateurs, charge les bornes sur les coefficients et appelle les fonctions de résolution définies dans le fichier auxiliaire.

Listing 1 – Programme principal salem17.jl.

```
1
2 using DelimitedFiles
3 using JuMP, Gurobi, Polynomials, Nemo
4
5 include("mesFonctionsSalem.jl")
6
7
8
9
10
11
12 R, x = polynomial_ring(ZZ, "x")
13
14 d = 17;
15
16 t1 = Polynomials.roots(Polynomial(reverse([1, 3, 1])))[2]
17 t2 = Polynomials.roots(Polynomial(reverse([1, 4, 3, -1])))[3]
18 t3 = Polynomials.roots(Polynomial(reverse([1,5,4,-10,-11])))[3]
19 t4 = Polynomials.roots(Polynomial(reverse([1,5,5,-5,-5])))[4]
20
21 aa = readdlm("lb17Salem.dat")
22 bb = readdlm("ub17Salem.dat")
23
24 aa = Int64.(aa)
25 bb = Int64.(bb)
26
27 aa2 = 0
28 bb2 = 0
29
30 u_tmp = 0
31
32 for k = 1:9000000000
33
34     global qqch = false
35
36     l = zeros(d);
37
38     while (abs(sum(l)-31) > 0.1) || abs(prod(l)-1) > 0.1
39         l = vcat(4.0*rand(d-1),4.0+1.8*rand())
40     end
41
42     l = sort(l)
43
44     l -= 3.0*ones(d)
45
46     r = zeros(d+1)
47     r[1], r[end-1], r[end] = -3.0, 1.0, 2.8
48     r[2:end-2] = 0.5*(l[1:end-2]+l[2:end-1])
49     r = sort(r)
50
51     solution(r,aa,bb)
```

52

53 `end`

B Fonctions de résolution : mesFonctionsSalem.jl

Le fichier suivant contient les fonctions qui construisent les modèles de programmation linéaire en nombres entiers. Chaque fonction crée un modèle Gurobi, impose les contraintes de coefficients et d'alternance de signes, puis vérifie les polynômes obtenus.

Listing 2 – Fonctions de résolution mesFonctionsSalem.jl.

```
1
2 function solution(r,aa,bb)
3     lpModel = Model(Gurobi.Optimizer)
4     set_optimizer_attribute(lpModel, "IntFeasTol", 1e-9)
5     set_optimizer_attribute(lpModel, "OptimalityTol", 1e-9)
6     set_optimizer_attribute(lpModel, "FeasibilityTol", 1e-9)
7     set_optimizer_attribute(lpModel, "TimeLimit", 1200)
8
9     JuMP.@variable(lpModel, p[1:d + 1], Int)
10
11     @constraint(lpModel, p[1] == 1)
12     @constraint(lpModel, p[2] == 20)
13     @constraint(lpModel, [i=3:d+1], aa[i] <= p[i] <= bb[i])
14     @constraint(lpModel, Polynomial(reverse(p))(-3.0) <= -0.9)
15     @constraint(lpModel, [i=2:d+1], Polynomial(reverse(p))(r[i]) * (-1)^(i+1) <=
        -1e-6)
16
17     # @constraint(lpModel, Polynomial(reverse(p))(-2.0) <= -0.9)
18
19     JuMP.optimize!(lpModel)
20
21     if termination_status(lpModel) == MOI.OPTIMAL
22         solution_coeffs = [Int64(round(JuMP.value(p[i]))) for i=1:d+1]
23         pp = sum(reverse(solution_coeffs)[i] * x^(i-1) for i in 1:length(
            solution_coeffs))
24         if isreal(Polynomials.roots(Polynomial(reverse(solution_coeffs))))
25             global qqch = true
26             f2=open("resSalem17Gen1.txt","a+");
27             println(f2, join(solution_coeffs, " "))
28             close(f2)
29             global u_tmp = solution_coeffs
30             solutionm(r,aa,bb)
31             solutionp(r,aa,bb)
32         end
33         if is_irreducible(pp) && isreal(Polynomials.roots(Polynomial(reverse(
            solution_coeffs))))
34             f1=open("resulSalem17Gen1.txt","a+");
35             println(f1, join(solution_coeffs, " "))
36             close(f1)
37         end
38     end
39     if termination_status(lpModel) == MOI.OPTIMAL
40         println("Solution trouvée : ", JuMP.value.(p))
41     else
42         println("Aucune solution trouvée. Statut de terminaison : ",
            termination_status(lpModel))
43     end
44 end
45
46
47
```

```

48 function solutionm(r,aa,bb)
49     lpModel = Model(Gurobi.Optimizer)
50     set_optimizer_attribute(lpModel, "IntFeasTol", 1e-9)
51     set_optimizer_attribute(lpModel, "OptimalityTol", 1e-9)
52     set_optimizer_attribute(lpModel, "FeasibilityTol", 1e-9)
53     set_optimizer_attribute(lpModel, "TimeLimit", 1200)
54
55     JuMP.@variable(lpModel, p[1:d + 1], Int)
56
57     @constraint(lpModel, p[1] == 1)
58     @constraint(lpModel, p[2] == 20)
59     @constraint(lpModel, [i=3:d+1], aa[i] <= p[i] <= bb[i])
60     @constraint(lpModel, Polynomial(reverse(p))(-3.0) <= -0.9)
61     @constraint(lpModel, [i=2:d+1], Polynomial(reverse(p))(r[i]) * (-1)^(i+1) <=
-1e-6)
62
63     @constraint(lpModel, Polynomial(reverse(p))(-2.0) <= -0.9)
64
65     JuMP.optimize!(lpModel)
66
67     if termination_status(lpModel) == MOI.OPTIMAL
68         solution_coeffs = [Int64(round(JuMP.value(p[i]))) for i=1:d+1]
69         pp = sum(reverse(solution_coeffs)[i] * x^(i-1) for i in 1:length(
solution_coeffs))
70         if isreal(Polynomials.roots(Polynomial(reverse(solution_coeffs))))
71             global qqch = true
72             f2=open("resSalem17Gen2.txt","a+");
73             println(f2, join(solution_coeffs, " "))
74             close(f2)
75             global u_tmp = solution_coeffs
76             solutionmm(r,aa,bb)
77             solutionmp(r,aa,bb)
78             solutionpm(r,aa,bb)
79             solutionpp(r,aa,bb)
80         end
81         if is_irreducible(pp) && isreal(Polynomials.roots(Polynomial(reverse(
solution_coeffs))))
82             f1=open("resulSalem17Gen2.txt","a+");
83             println(f1, join(solution_coeffs, " "))
84             close(f1)
85         end
86     end
87     if termination_status(lpModel) == MOI.OPTIMAL
88         println("Solution trouvée : ", JuMP.value.(p))
89     else
90         println("Aucune solution trouvée. Statut de terminaison : ",
termination_status(lpModel))
91     end
92 end
93
94 function solutionp(r,aa,bb)
95     lpModel = Model(Gurobi.Optimizer)
96     set_optimizer_attribute(lpModel, "IntFeasTol", 1e-9)
97     set_optimizer_attribute(lpModel, "OptimalityTol", 1e-9)
98     set_optimizer_attribute(lpModel, "FeasibilityTol", 1e-9)
99     set_optimizer_attribute(lpModel, "TimeLimit", 1200)
100
101     JuMP.@variable(lpModel, p[1:d + 1], Int)

```

```

102
103 @constraint(lpModel, p[1] == 1)
104 @constraint(lpModel, p[2] == 20)
105 @constraint(lpModel, [i=3:d+1], aa[i] <= p[i] <= bb[i])
106 @constraint(lpModel, Polynomial(reverse(p))(-3.0) <= -0.9)
107 @constraint(lpModel, [i=2:d+1], Polynomial(reverse(p))(r[i]) * (-1)^(i+1) <=
-1e-6)
108
109 @constraint(lpModel, Polynomial(reverse(p))(-2.0) >= 0.9)
110
111 JuMP.optimize!(lpModel)
112
113 if termination_status(lpModel) == MOI.OPTIMAL
114     solution_coeffs = [Int64(round(JuMP.value(p[i]))) for i=1:d+1]
115     pp = sum(reverse(solution_coeffs)[i] * x^(i-1) for i in 1:length(
solution_coeffs))
116     if isreal(Polynomials.roots(Polynomial(reverse(solution_coeffs))))
117         global qqch = true
118         f2=open("resSalem17.Gen2txt","a+");
119         println(f2, join(solution_coeffs, " "))
120         close(f2)
121         global u_tmp = solution_coeffs
122         solutionmm(r,aa,bb)
123         solutionmp(r,aa,bb)
124         solutionpm(r,aa,bb)
125         solutionpp(r,aa,bb)
126     end
127     if is_irreducible(pp) && isreal(Polynomials.roots(Polynomial(reverse(
solution_coeffs))))
128         f1=open("resulSalem17Gen2.txt","a+");
129         println(f1, join(solution_coeffs, " "))
130         close(f1)
131     end
132 end
133 if termination_status(lpModel) == MOI.OPTIMAL
134     println("Solution trouvée : ", JuMP.value.(p))
135 else
136     println("Aucune solution trouvée. Statut de terminaison : ",
termination_status(lpModel))
137 end
138 end
139
140
141
142
143 function solutionmm(r,aa,bb)
144     lpModel = Model(Gurobi.Optimizer)
145     set_optimizer_attribute(lpModel, "IntFeasTol", 1e-9)
146     set_optimizer_attribute(lpModel, "OptimalityTol", 1e-9)
147     set_optimizer_attribute(lpModel, "FeasibilityTol", 1e-9)
148     set_optimizer_attribute(lpModel, "TimeLimit", 1200)
149
150     JuMP.@variable(lpModel, p[1:d + 1], Int)
151
152     @constraint(lpModel, p[1] == 1)
153     @constraint(lpModel, p[2] == 20)
154     @constraint(lpModel, [i=3:d+1], aa[i] <= p[i] <= bb[i])
155     @constraint(lpModel, Polynomial(reverse(p))(-3.0) <= -0.9)

```

```

156 @constraint(lpModel, [i=2:d+1], Polynomial(reverse(p))(r[i]) * (-1)^(i+1) <=
-1e-6)
157
158 @constraint(lpModel, Polynomial(reverse(p))(-2.0) <= -0.9)
159 @constraint(lpModel, Polynomial(reverse(p))(t1) <= -1e-6)
160
161 JuMP.optimize!(lpModel)
162
163 if termination_status(lpModel) == MOI.OPTIMAL
164     solution_coeffs = [Int64(round(JuMP.value(p[i]))) for i=1:d+1]
165     pp = sum(reverse(solution_coeffs)[i] * x^(i-1) for i in 1:length(
solution_coeffs))
166     if isreal(Polynomials.roots(Polynomial(reverse(solution_coeffs))))
167         global qqch = true
168         f2=open("resSalem17Gen3.txt","a+");
169         println(f2, join(solution_coeffs, " "))
170         close(f2)
171         global u_tmp = solution_coeffs
172     end
173     if is_irreducible(pp) && isreal(Polynomials.roots(Polynomial(reverse(
solution_coeffs))))
174         f1=open("resulSalem17Gen3.txt","a+");
175         println(f1, join(solution_coeffs, " "))
176         close(f1)
177     end
178 end
179 if termination_status(lpModel) == MOI.OPTIMAL
180     println("Solution trouvée : ", JuMP.value.(p))
181 else
182     println("Aucune solution trouvée. Statut de terminaison : ",
termination_status(lpModel))
183 end
184 end
185
186 function solutionmp(r,aa,bb)
187     lpModel = Model(Gurobi.Optimizer)
188     set_optimizer_attribute(lpModel, "IntFeasTol", 1e-9)
189     set_optimizer_attribute(lpModel, "OptimalityTol", 1e-9)
190     set_optimizer_attribute(lpModel, "FeasibilityTol", 1e-9)
191     set_optimizer_attribute(lpModel, "TimeLimit", 1200)
192
193     JuMP.@variable(lpModel, p[1:d + 1], Int)
194
195     @constraint(lpModel, p[1] == 1)
196     @constraint(lpModel, p[2] == 20)
197     @constraint(lpModel, [i=3:d+1], aa[i] <= p[i] <= bb[i])
198     @constraint(lpModel, Polynomial(reverse(p))(-3.0) <= -0.9)
199     @constraint(lpModel, [i=2:d+1], Polynomial(reverse(p))(r[i]) * (-1)^(i+1) <=
-1e-6)
200
201     @constraint(lpModel, Polynomial(reverse(p))(-2.0) <= -0.9)
202     @constraint(lpModel, Polynomial(reverse(p))(t1) >= 1e-6)
203
204     JuMP.optimize!(lpModel)
205
206     if termination_status(lpModel) == MOI.OPTIMAL
207         solution_coeffs = [Int64(round(JuMP.value(p[i]))) for i=1:d+1]
208         pp = sum(reverse(solution_coeffs)[i] * x^(i-1) for i in 1:length(

```

```

solution_coeffs))
209     if isreal(Polynomial.roots(Polynomial(reverse(solution_coeffs))))
210         global qqch = true
211         f2=open("resSalem17Gen3.txt","a+");
212         println(f2, join(solution_coeffs, " "))
213         close(f2)
214         global u_tmp = solution_coeffs
215     end
216     if is_irreducible(pp) && isreal(Polynomial.roots(Polynomial(reverse(
solution_coeffs))))
217         f1=open("resulSalem17Gen3.txt","a+");
218         println(f1, join(solution_coeffs, " "))
219         close(f1)
220     end
221 end
222 if termination_status(lpModel) == MOI.OPTIMAL
223     println("Solution trouvée : ", JuMP.value.(p))
224 else
225     println("Aucune solution trouvée. Statut de terminaison : ",
termination_status(lpModel))
226 end
227 end
228
229
230
231 function solutionpm(r,aa,bb)
232     lpModel = Model(Gurobi.Optimizer)
233     set_optimizer_attribute(lpModel, "IntFeasTol", 1e-9)
234     set_optimizer_attribute(lpModel, "OptimalityTol", 1e-9)
235     set_optimizer_attribute(lpModel, "FeasibilityTol", 1e-9)
236     set_optimizer_attribute(lpModel, "TimeLimit", 1200)
237
238     JuMP.@variable(lpModel, p[1:d + 1], Int)
239
240     @constraint(lpModel, p[1] == 1)
241     @constraint(lpModel, p[2] == 20)
242     @constraint(lpModel, [i=3:d+1], aa[i] <= p[i] <= bb[i])
243     @constraint(lpModel, Polynomial(reverse(p))(-3.0) <= -0.9)
244     @constraint(lpModel, [i=2:d+1], Polynomial(reverse(p))(r[i]) * (-1)^(i+1) <=
-1e-6)
245
246     @constraint(lpModel, Polynomial(reverse(p))(-2.0) >= 0.9)
247     @constraint(lpModel, Polynomial(reverse(p))(t1) <= -1e-6)
248
249     JuMP.optimize!(lpModel)
250
251     if termination_status(lpModel) == MOI.OPTIMAL
252         solution_coeffs = [Int64(round(JuMP.value(p[i]))) for i=1:d+1]
253         pp = sum(reverse(solution_coeffs)[i] * x^(i-1) for i in 1:length(
solution_coeffs))
254         if isreal(Polynomial.roots(Polynomial(reverse(solution_coeffs))))
255             global qqch = true
256             f2=open("resSalem17Gen3.txt","a+");
257             println(f2, join(solution_coeffs, " "))
258             close(f2)
259             global u_tmp = solution_coeffs
260         end
261         if is_irreducible(pp) && isreal(Polynomial.roots(Polynomial(reverse(

```

```

solution_coeffs))))
262         f1=open("resulSalem17Gen3.txt","a+");
263         println(f1, join(solution_coeffs, " "))
264         close(f1)
265     end
266 end
267 if termination_status(lpModel) == MOI.OPTIMAL
268     println("Solution trouvée : ", JuMP.value.(p))
269 else
270     println("Aucune solution trouvée. Statut de terminaison : ",
termination_status(lpModel))
271 end
272 end
273
274 function solutionpp(r,aa,bb)
275     lpModel = Model(Gurobi.Optimizer)
276     set_optimizer_attribute(lpModel, "IntFeasTol", 1e-9)
277     set_optimizer_attribute(lpModel, "OptimalityTol", 1e-9)
278     set_optimizer_attribute(lpModel, "FeasibilityTol", 1e-9)
279     set_optimizer_attribute(lpModel, "TimeLimit", 1200)
280
281     JuMP.@variable(lpModel, p[1:d + 1], Int)
282
283     @constraint(lpModel, p[1] == 1)
284     @constraint(lpModel, p[2] == 20)
285     @constraint(lpModel, [i=3:d+1], aa[i] <= p[i] <= bb[i])
286     @constraint(lpModel, Polynomial(reverse(p))(-3.0) <= -0.9)
287     @constraint(lpModel, [i=2:d+1], Polynomial(reverse(p))(r[i]) * (-1)^(i+1) <=
-1e-6)
288
289     @constraint(lpModel, Polynomial(reverse(p))(-2.0) >= 0.9)
290     @constraint(lpModel, Polynomial(reverse(p))(t1) >= 1e-6)
291
292     JuMP.optimize!(lpModel)
293
294     if termination_status(lpModel) == MOI.OPTIMAL
295         solution_coeffs = [Int64(round(JuMP.value(p[i]))) for i=1:d+1]
296         pp = sum(reverse(solution_coeffs)[i] * x^(i-1) for i in 1:length(
solution_coeffs))
297         if isreal(Polynomials.roots(Polynomial(reverse(solution_coeffs))))
298             global qqch = true
299             f2=open("resSalem17Gen3.txt","a+");
300             println(f2, join(solution_coeffs, " "))
301             close(f2)
302             global u_tmp = solution_coeffs
303         end
304         if is_irreducible(pp) && isreal(Polynomials.roots(Polynomial(reverse(
solution_coeffs))))
305             f1=open("resulSalem17Gen3.txt","a+");
306             println(f1, join(solution_coeffs, " "))
307             close(f1)
308         end
309     end
310     if termination_status(lpModel) == MOI.OPTIMAL
311         println("Solution trouvée : ", JuMP.value.(p))
312     else
313         println("Aucune solution trouvée. Statut de terminaison : ",
termination_status(lpModel))

```

```
$14     end
$15 end
```

Références

- [1] D. H. Lehmer, *Factorization of certain cyclotomic functions*, Annals of Mathematics 34 (1933), 461–479.
- [2] C. Smyth, *Seventy years of Salem numbers*, Bulletin of the London Mathematical Society 47 (2015), no. 3, 379–395.
- [3] C. Smyth, *Salem numbers of negative trace*, Mathematics of Computation 69 (2000), no. 230, 827–838.
- [4] J. F. McKee and C. J. Smyth, *Salem numbers of trace -2 and traces of totally positive algebraic integers*, Lecture Notes in Computer Science 3076, Springer, 2004, 327–337.
- [5] J. F. McKee and C. J. Smyth, *There are Salem numbers of every trace*, Bulletin of the London Mathematical Society 37 (2005), no. 1, 25–36.
- [6] J. F. McKee, *Computing totally positive algebraic integers of small trace*, Mathematics of Computation 80, no. 274 (2011), 1041–1052.
- [7] S. El Otmani, A. Maul, G. Rhin and J.-M. Sac-Épée, *Finding degree-16 monic irreducible integer polynomials of minimal trace by optimization methods*, Experimental Mathematics 23 (2014), no. 1, 1–5.
- [8] J. F. McKee, *Computing totally positive algebraic integers of small trace*, Mathematics of Computation 80 (2011), no. 274, 1041–1052.
- [9] S. El Otmani, G. Rhin and J.-M. Sac-Épée, *A Salem number with degree 34 and trace -3* , Journal of Number Theory 150 (2015), 21–25.
- [10] C. Wang, J. Wu and Q. Wu, *Totally positive algebraic integers with small trace*, Mathematics of Computation 90, no. 331 (2021), 2317–2332.
- [11] J.-M. Sac-Épée, *Small degree Salem numbers with trace -3* , Control and Cybernetics 52, no. 3 (2023), 335–346.
- [12] J.-M. Sac-Épée, *Salem numbers less than $49/37$* , à paraître dans Rocky Mountain Journal of Mathematics